

Opacity Is Not Just Opacity

Chunran Zhang 

School of Computing and Artificial Intelligence
Southwest Jiaotong University
Chengdu, China
chronis@my.swjtu.edu.cn

Abstract

Opacity is commonly understood as how much an object obscures its background, with full opacity as the endpoint. Yet alpha compositing scales the color difference between the object and its background, with transparency corresponding to contraction. At $\alpha = 1$, the output reaches the source color, but difference scaling need not stop there. We retain the compositing equation and extend the coefficient domain from $[0, 1]$ to $[0, +\infty)$, adding expansion while preserving contraction. Each object carries a fixed source color and coefficient; the actual background determines the direction of enhancement. Contrast enhancement that adapts to the background thus becomes part of the object’s compositing properties. The same graphical object can be reused across web contexts, reducing the design and maintenance of separate color variants. Implementation reuses the original equation without increasing the arithmetic operation count within the same pipeline. Across all 8-bit sRGB source colors and 16 predefined light and dark backgrounds, a fixed $\alpha = 1.1$ increases the contrast ratio in 99.8145% of combinations. It also moves 4.8346% of all combinations from below 3 : 1 to at least 3 : 1, meeting this contrast threshold without changing their source colors. Coefficient sweeps and background stress tests identify conditions under which clipping reduces color separation or the contrast ratio decreases. Output validation and timing across three browsers demonstrate implementation in a common WebGL pipeline, with no sustained runtime increase for $\alpha > 1$.

Keywords

alpha compositing, color contrast, background adaptation, web graphics

1 Introduction

Opacity is commonly understood as the degree to which a graphical object obscures its background. In color compositing, it specifies the weight of the source color’s contribution [7]. That weight alone does not determine the displayed color. Lowering opacity does not specify whether a color becomes lighter or darker; the change depends on the other color participating in the composite.

A fully transparent region contributes no other color. This absence, denoted here by null, is neither white nor black. When a source color C_s is composited over such a region with $0 < \alpha \leq 1$, the premultiplied output and output alpha are αC_s and α , respectively. Recovering the non-premultiplied color gives [7, 10]

$$C_o = \frac{\alpha C_s}{\alpha} = C_s.$$

The contribution weight decreases, but the source color is retained. Once a background contributes, both colors determine the

displayed result. Alpha can therefore be understood as a parameter of the relationship between an object and its background.

Let C_b denote the color of the canvas after its background layers have been composited, C_s the foreground source color, and α its opacity. With the canvas opaque, the displayed result is

$$C_o = \alpha C_s + (1 - \alpha) C_b.$$

Rewriting this equation as

$$C_o - C_b = \alpha(C_s - C_b)$$

shows that alpha scales the difference between the object and its background. Transparency is the effect of contracting that difference.

Within $[0, 1]$, a smaller coefficient moves the output closer to the background; at one, the output reaches the source color. Restricting the coefficient to $[0, 1]$ therefore restricts this relationship to contraction. This upper bound matches the meaning of full opacity, but is not required by difference scaling itself.

We retain the compositing equation and additionally allow $\alpha \in (1, +\infty)$. The output then moves beyond the source color, expanding the difference in the direction away from the background. The existing compositing property preserves contraction and adds expansion: the same relationship can blend an object into its background or enhance its difference from it. Opacity describes the property’s role within the conventional range, whereas scaling the object-background difference permits a broader domain. Separating these interpretations allows the same property to support contrast enhancement while retaining its transparency effects.

With the source color and coefficient fixed, the actual background still participates in compositing and determines the direction of expansion. The object’s definition remains unchanged while its displayed output adjusts to the background. Contrast enhancement that adapts to the background thus becomes part of the object’s compositing properties. In a rendering pipeline supporting the extended coefficient, the application supplies the resolved background color. The same icon or graphic can then carry its parameters across pages, themes, and embedded canvases.

Specifying colors separately for each background requires storing and maintaining those choices after design changes. Assigning adaptation to the compositing relationship concentrates design on the object’s source color and coefficient. Wherever this fixed configuration meets the target contrast ratio, it removes the need to specify a separate output color. Design changes can then be applied to the shared object definition, with its outputs recomputed and checked across the target backgrounds.

This capability is obtained by extending the domain of the existing compositing coefficient. The source color, background color, and coefficient still enter the original equation. No separate color-search

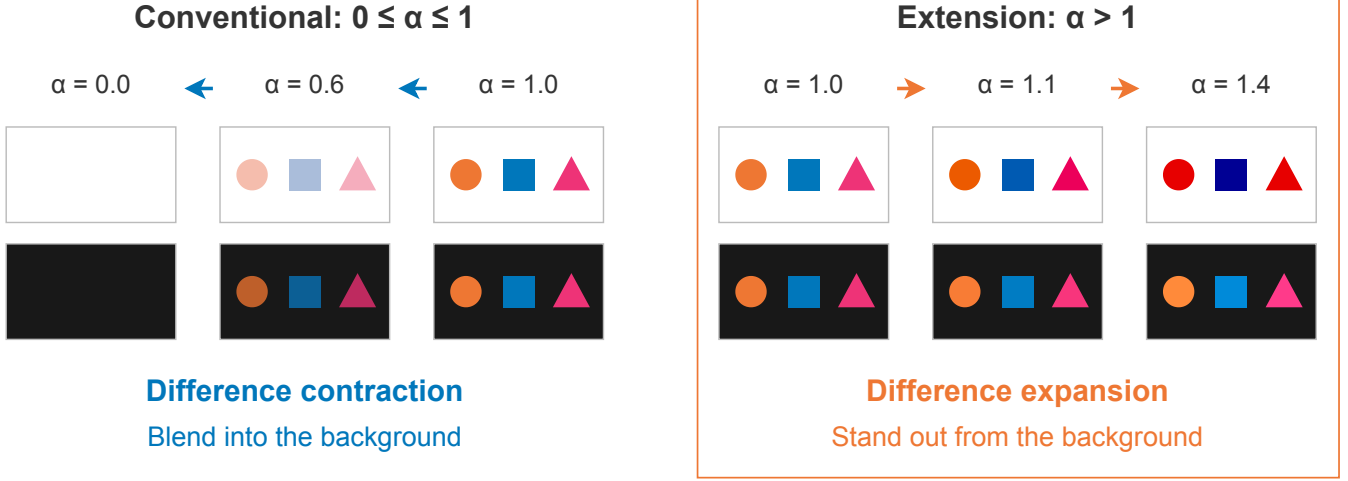


Figure 1: The same source colors composited on light and dark canvases. The left panel shows contraction within the conventional range; the outlined right panel shows the added expansion range. At $\alpha = 1$, the source colors are preserved.

or transformation stage is introduced; within the same compositing pipeline supporting the extension, the per-pixel arithmetic operation count remains unchanged. Fixed representation, adaptation to the background, and reuse of the computation follow from the same relationship. We evaluate contrast enhancement on common canvases, examine color-separation costs and background coverage through coefficient sweeps and stress tests, and compare runtime across the conventional and extended ranges.

2 Related Work

Contrast requirements for graphical objects reused across backgrounds are often addressed by changing their colors. Microsoft’s icon guidelines allow separate assets for light and dark themes, storing the adaptation as different versions [3]. Iris selects replacement colors from interface context to repair text and image contrast in Android apps [11]. The former requires designing, checking, and maintaining multiple versions, with consistency preserved after changes to the object. The latter identifies the attributes to repair and performs context-dependent color selection. Adaptation therefore involves maintaining mappings alongside the object design or performing an additional repair process.

Color compositing already allows a fixed object to produce different outputs as the background changes. W3C’s Difference and Exclusion blend modes each specify how to compute such outputs [7]. We retain the ordinary alpha compositing expression and extend the coefficient domain from $[0, 1]$ to $[0, +\infty)$, adding expansion while preserving contraction. Conventional transparency effects and background contrast enhancement are expressed by different values of the same coefficient in the same equation.

Willis’s Projective Alpha Colour interprets alpha through color energy and carrying area, including negative values and values above one [10]. In its reflection formulation, normalization recovers the same color for any nonzero alpha; its compositing extensions introduce filter and illumination interactions. We study a specific use

of source-over compositing: a fixed source color and coefficient produce contrast enhancement relative to the actual background. We establish which background differences survive clipping and evaluate contrast gains, color separation, and runtime. These properties and measurements characterize the use of the extended coefficient for fixed-object contrast enhancement.

Separate assets store background-specific color choices, while repair methods select replacements from context. Here, the actual background enters the original compositing equation, and the object’s existing coefficient specifies enhancement strength. On backgrounds where the output meets the target contrast ratio, the shared object definition removes the need for a separately specified color. The original expression supplies this adaptation without adding object parameters or per-pixel arithmetic operations within the same pipeline.

3 Method

All color calculations and property analyses in this section use real-valued linear RGB. Figure 1 summarizes the conventional and extended coefficient ranges.

3.1 Compositing

Let C_s and C_b be the non-premultiplied source and background colors, with alpha values $\alpha_s, \alpha_b \in [0, 1]$. Standard source-over compositing gives [4, 7]

$$\begin{aligned} c_o &= \alpha_s C_s + (1 - \alpha_s) \alpha_b C_b, \\ \alpha_o &= \alpha_s + (1 - \alpha_s) \alpha_b. \end{aligned} \quad (1)$$

For $\alpha_o > 0$, the output color is $C_o = c_o / \alpha_o$. Once background layers are composited onto the final opaque canvas, $\alpha_b = 1$, giving

$$C_o = \alpha_s C_s + (1 - \alpha_s) C_b. \quad (2)$$

3.2 From Transparency to Expansion of Color Differences

In the conventional domain, $\alpha = 1$ means full opacity, with the output equal to the source color. Writing the relationship between output and background as

$$C_o - C_b = \alpha(C_s - C_b) \quad (3)$$

shows that the source color corresponds to a difference-scaling coefficient of one. Restricting the coefficient to $[0, 1]$ allows the difference only to remain unchanged or contract; the equation does not require scaling to stop there.

We extend the coefficient domain to $\alpha \in [0, +\infty)$ while retaining the original expression:

$$C_{\text{raw}} = \alpha C_s + (1 - \alpha)C_b. \quad (4)$$

Its difference from the background satisfies

$$C_{\text{raw}} - C_b = \alpha(C_s - C_b). \quad (5)$$

For $\alpha \in [0, 1)$, the difference contracts; at $\alpha = 1$, the source color is preserved; for $\alpha \in (1, +\infty)$, the difference expands. If $C_s = C_b$, every coefficient gives $C_{\text{raw}} = C_b$: a zero difference cannot be amplified.

Relative luminance satisfies the same relationship:

$$Y_{\text{raw}} - Y_b = \alpha(Y_s - Y_b). \quad (6)$$

Thus, when source and background luminances differ and the output remains in gamut, $\alpha > 1$ increases the contrast ratio.

Per-channel clipping to $[0, 1]$ gives the final output:

$$C_o = \text{clip}_{[0,1]}(C_{\text{raw}}). \quad (7)$$

Within the conventional range $\alpha \in [0, 1]$, clipping leaves the result unchanged, preserving ordinary compositing in full. The same compositing property thus adds expansion while preserving its transparency effects.

3.3 Background Contrast Enhancement with Fixed Objects

Expansion takes the actual background as its reference. For a fixed source color C_s and coefficient α , a change in the background produces

$$\Delta C_{\text{raw}} = (1 - \alpha)\Delta C_b. \quad (8)$$

For $\alpha > 1$, the unclipped output changes in the opposite direction to the background. Fixed C_s and α therefore specify an enhancement rule whose direction adapts to the actual background. The application supplies the resolved C_b at compositing time. With the same color space and clipping pipeline, both coefficient ranges execute the same expression, preserving the per-pixel arithmetic operation count. This adaptation uses the object's existing parameters and the original computation, with no additional color-search or optimization stage.

3.4 Enhancement Strength, Color Separation, and Background Differences

The coefficient in a fixed configuration controls the extent of expansion. For two objects using the same coefficient on the same background, their unclipped outputs satisfy

$$C_{\text{raw},i} - C_{\text{raw},j} = \alpha(C_{s,i} - C_{s,j}). \quad (9)$$

The common background term cancels, so the difference between objects scales by the same factor. For $\alpha > 1$, the same coefficient therefore expands both object-background and inter-object RGB differences before clipping.

Clipping can nevertheless reduce color separation or map distinct source colors to the same output. Lowering the coefficient reduces clipping, controlling the tradeoff between background contrast enhancement and color separation between objects. The coefficient thus determines both the enhancement strength and the color-separation cost of a fixed configuration.

This convergence occurs between objects; their differences from the background are preserved. For $\alpha \in [1, +\infty)$, the final output, including clipping, satisfies

$$|C_{o,k} - C_{b,k}| \geq |C_{s,k} - C_{b,k}|. \quad (10)$$

Consequently, for any $1 \leq p \leq \infty$,

$$\|C_o - C_b\|_p \geq \|C_s - C_b\|_p. \quad (11)$$

The RGB distance from the output to the background does not decrease. On black and white backgrounds, this per-channel guarantee also directly implies a nondecreasing contrast ratio. On black backgrounds, $C_{o,k} \geq C_{s,k}$ in every channel, so relative luminance does not decrease. On white backgrounds, $C_{o,k} \leq C_{s,k}$ in every channel, so relative luminance does not increase. Both cases preserve or increase the contrast ratio against the background, including after clipping.

4 Experiments

4.1 Experimental Setup

We fix the object's source color and coefficient to evaluate the same compositing relationship across backgrounds, measuring contrast enhancement, color separation between objects, and web rendering runtime.

The numerical evaluation enumerates all 256^3 8-bit sRGB source colors on 16 black, white, near-black, near-white, and mildly tinted canvases. A 65^3 source grid is used to sweep $\alpha \in [0, 2]$ in steps of 0.025. A further evaluation combines 17^3 source colors with 33^3 background colors to broaden background coverage. These canvases and grids are synthetic test conditions, not a sample of color usage on the web.

All configurations use linear RGB and per-channel clipping, with $\alpha = 1$ as the reference. We measure WCAG contrast-ratio changes [8], using Difference and Exclusion as controls in the same color space [7]. Color separation is measured by CIEDE2000 [5] and color collisions, defined as identical 8-bit outputs. We test all distinct pairs from Tol's seven-color Vibrant palette [6] and 65,536 distinct source-color pairs generated with a fixed seed. Appendix A details the metrics, backgrounds, and rendering.

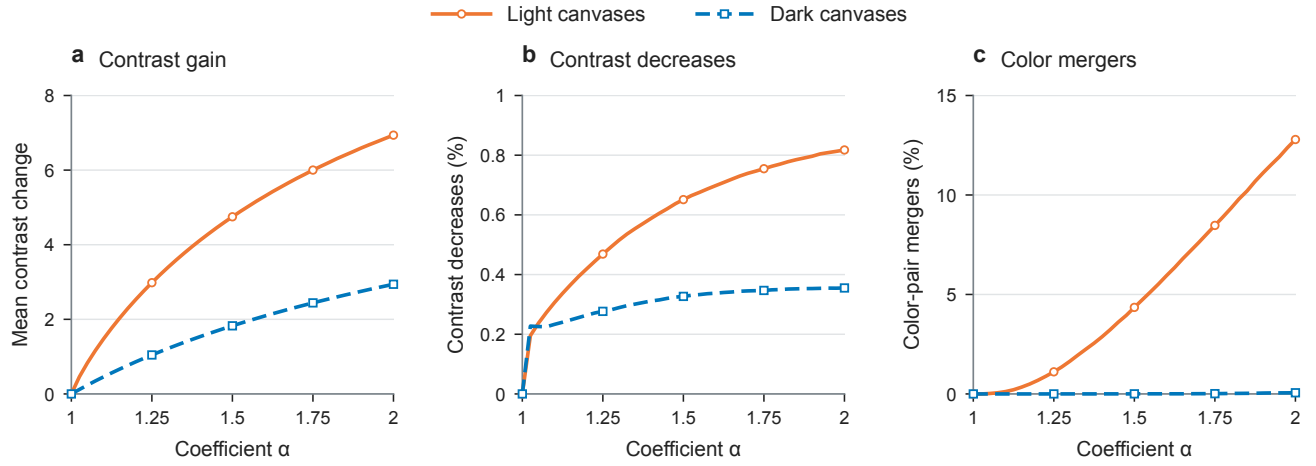


Figure 3: Coefficient effects on enhancement and its costs. (a) Mean change in contrast ratio; (b) proportion of contrast-ratio decreases; (c) 8-bit color-collision rate. Each group weights eight canvases equally. Panels (a-b) use 65^3 source colors; panel (c) uses 65,536 distinct source-color pairs. All 41 measurements in $[1, 2]$ are plotted.

4.2 Background Contrast Enhancement with a Fixed Configuration

The same source icons exhibit two roles of the compositing coefficient on light and dark canvases. Values $\alpha < 1$ blend the icons into the background, whereas $\alpha > 1$ expands their color differences from it (Figure 2). Each configuration fixes the source colors and coefficient, with the actual background determining the displayed output.

Table 1: Contrast-ratio changes under different background coverage. The predefined-canvas evaluation enumerates all 8-bit source colors; the broader evaluation uses 17^3 source colors and 33^3 backgrounds. Remaining combinations are unchanged within numerical tolerance.

α	Predefined (%)		Broad grid (%)	
	Increase	Decrease	Increase	Decrease
1.1	99.8145	0.1853	90.3672	9.3318
1.4	99.6132	0.3867	86.7022	12.9968

Across all source colors and the 16 canvases, $\alpha = 1.1$ increases the contrast ratio in 99.8145% of combinations, with a mean increase of 0.9783. Under the same conditions, the mean changes for Difference and Exclusion are -1.3050 and -1.3104 , respectively.

At this coefficient, 4.8346% of all source-canvas combinations cross from a contrast ratio below 3 : 1 to at least 3 : 1. These combinations reach the threshold while retaining their source colors, reducing the need for replacement colors to meet this contrast requirement.

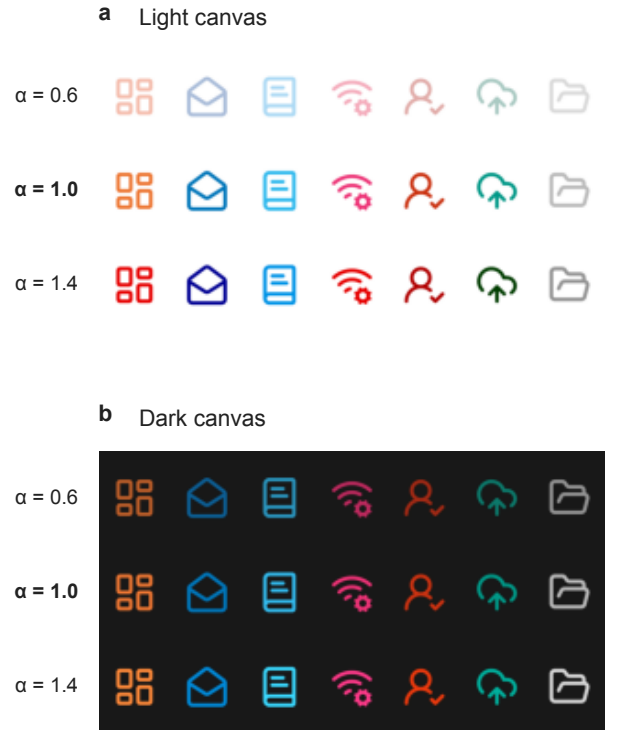


Figure 2: Fixed icons on light and dark canvases. Rows correspond to $\alpha = 0.6, 1.0, 1.4$; each column retains the same icon and source color. These illustrations are rendered at 192 px.

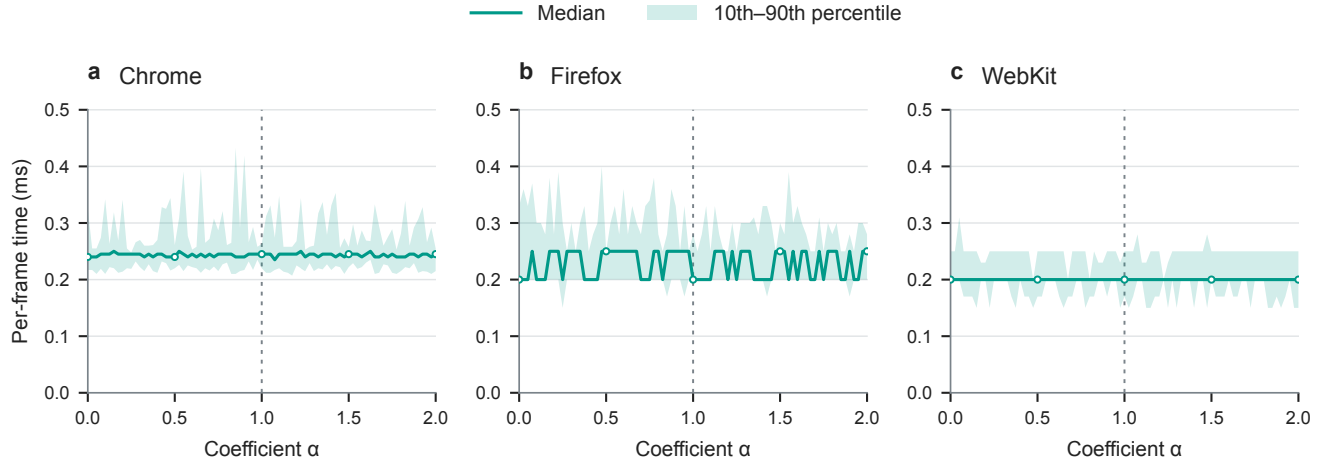


Figure 5: Runtime across the full coefficient range. Lines show the median of the block means; bands show the empirical 10th–90th percentiles. The vertical dotted line marks $\alpha = 1$. Each frame draws 300 icons and synchronously reads back one pixel. Firefox and WebKit block timings exhibit whole-millisecond steps.

Black and white canvases show no contrast-ratio decreases; other predefined canvases admit decreases. Their frequency increases on the broader background grid (Table 1).

On the broader grid, the maximum contrast-ratio losses are 0.4655 and 1.6394 at $\alpha = 1.1$ and 1.4, respectively. Among combinations with decreases, 99.34% and 81.53%, respectively, have losses no greater than 0.25. Loss denotes the absolute decrease in the contrast ratio.

Both coefficients incur their largest loss for a dark blue object on a yellow background: the source is sRGB (0, 0, 0.8125) and the background is (1, 1, 0). The original contrast ratio of 10.2773 falls to 9.8118 at $\alpha = 1.1$ and 8.6379 at $\alpha = 1.4$. Extrapolation drives the red and green channels below zero, where they are clipped, while the blue channel continues to increase. The object’s luminance consequently moves closer to the background luminance. Nondecreasing per-channel background differences can therefore coexist with a decreasing contrast ratio.

4.3 Enhancement Strength and Color Separation

Background contrast enhancement must also account for color separation between objects. Increasing the coefficient raises the mean contrast-ratio gain, but also makes distinct source colors more likely to produce identical outputs (Figure 3).

Of the 336 Vibrant pair-background combinations, 32 have a smaller CIEDE2000 color difference at $\alpha = 1.1$, with no color collisions. At $\alpha = 1.4$, 88 combinations have smaller color differences, including eight collisions. Among the 1,048,576 random-pair-background combinations, collisions increase from 617 to 15,098.

Figure 4 shows the effects of clipping. As channels reach their limits, distinct source colors become less separated or coincide.

Lowering the coefficient reduces this convergence while reducing the extent of background contrast enhancement.



Figure 4: Outputs for all seven Vibrant source colors on a white canvas. Columns fix the source color and rows vary the coefficient.

The coefficient sweep characterizes the choices available for a fixed configuration. Given the source colors, a shared coefficient controls the tradeoff between background enhancement and

color separation, and the selected configuration can then be applied across canvases. The background participates in output computation, while the coefficient specifies the chosen enhancement strength.

Full icon arrays on near-source backgrounds are shown in Appendix A.4.

4.4 Web Implementation and Runtime

The WebGL implementation [1] uses the same compositing expression throughout. The object supplies the source color and coefficient, and the application-managed canvas supplies the background color. Geometric coverage is handled separately at antialiased edges. The conventional and extended ranges share the shader and rendering pipeline, differing only in the coefficient value.

Across 300 Lucide icons [2], three sizes, 16 canvases, and nine configurations, we generate 388,800 rendering instances across three browsers. For every pixel with nonzero coverage, the maximum channel error against the CPU reference is at most 1/255 in Chrome, Firefox, and WebKit. This validates implementation of the compositing relationship for the tested icons, sizes, and backgrounds.

On the same Apple M4 Pro device, each browser is timed over all 81 coefficients. Each coefficient has 25 measurement blocks of 20 consecutive frames. Coefficients are randomly ordered within each round, yielding 121,500 timed frames in total (Figure 5). Browsers are measured sequentially.

The median time across coefficients ranges from 0.235 to 0.250 ms in Chrome and from 0.200 to 0.250 ms in Firefox. WebKit has a median of 0.200 ms at every coefficient.

Within the tested rendering pipeline, the same compositing property adds contrast enhancement that adapts to the background while preserving its transparency effects. The computation steps remain unchanged, with no sustained additional runtime observed.

5 Conclusion

Interpreting alpha solely as opacity makes full opacity the end-point of its role. Yet alpha in the compositing equation scales the color difference between object and background, a relationship that permits coefficients above one. The restriction follows from the opacity interpretation, rather than the difference-scaling expression. We extend the coefficient domain from $[0, 1]$ to $[0, +\infty)$, adding expansion while preserving contraction. The source color and coefficient remain fixed, while the actual background determines the enhancement direction. Contrast enhancement that adapts to the background thereby becomes part of the object’s compositing properties.

Across all 8-bit source colors and 16 common canvases, a fixed $\alpha = 1.1$ increases the contrast ratio in 99.8145% of combinations. It also moves 4.8346% of all combinations from below 3 : 1 to at least 3 : 1. These combinations reach the threshold without changing the source color, identifying cases where the fixed configuration removes the need to specify a replacement color. Background stress tests and coefficient sweeps also reveal costs from contrast-ratio decreases and reduced color separation.

The change needed to obtain this capability concerns the coefficient domain; the original compositing expression remains unchanged. Within the same pipeline, the per-pixel arithmetic operation count does not increase, and browser tests show no sustained additional runtime. A property conventionally used to control transparency can thus support contrast enhancement that adapts to the background through a fixed representation and the original computation. This reduces the need for separate color design and maintenance across backgrounds. The usual meaning of opacity does not exhaust the role of this compositing relationship: *Opacity Is Not Just Opacity*.

Acknowledgments

OpenAI Codex assisted with manuscript drafting, translation, and editing, and with implementing the experimental code and preparing figures under the author’s direction.

References

- [1] Khronos Group. [n. d.]. WebGL Specification, Version 1.0. Accessed September 13, 2026. <https://registry.khronos.org/webgl/specs/latest/1.0/>
- [2] Lucide Contributors. [n. d.]. Lucide. lucide-static version 1.45.0. Accessed September 13, 2026. <https://lucide.dev/>
- [3] Microsoft. 2026. Design Guidelines for Windows App Icons. Updated March 7, 2026. Accessed September 13, 2026. <https://learn.microsoft.com/en-us/windows/apps/design/iconography/app-icon-design>
- [4] Thomas Porter and Tom Duff. 1984. Compositing Digital Images. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '84)*. ACM, New York, NY, USA, 253–259. doi:10.1145/800031.808606
- [5] Gaurav Sharma, Wencheng Wu, and Edul N. Dalal. 2005. The CIEDE2000 Color-Difference Formula: Implementation Notes, Supplementary Test Data, and Mathematical Observations. *Color Research & Application* 30, 1 (2005), 21–30. doi:10.1002/col.20070
- [6] Paul Tol. [n. d.]. Colour Schemes. Vibrant palette. Accessed September 13, 2026. <https://sronpersonalpages.nl/~pault/>
- [7] W3C. 2024. *Compositing and Blending Level 1*. Candidate Recommendation Draft. World Wide Web Consortium. Accessed September 13, 2026. <https://www.w3.org/TR/2024/CRD-compositing-1-20240321/>
- [8] W3C. 2024. *Web Content Accessibility Guidelines (WCAG) 2.2*. Recommendation. World Wide Web Consortium. Definition of contrast ratio. Accessed September 13, 2026. <https://www.w3.org/TR/WCAG22/>
- [9] W3C. 2026. *CSS Color Module Level 4*. Candidate Recommendation Draft. World Wide Web Consortium. sRGB and linear-light sRGB conversions. Accessed September 13, 2026. <https://www.w3.org/TR/2026/CRD-css-color-4-20260908/>
- [10] Philip Willis. 2006. Projective Alpha Colour. *Computer Graphics Forum* 25, 3 (2006), 557–566. doi:10.1111/j.1467-8659.2006.00975.x
- [11] Yuxin Zhang, Sen Chen, Lingling Fan, Chunyang Chen, and Xiaohong Li. 2023. Automated and Context-Aware Repair of Color-Related Accessibility Issues for Android Apps. arXiv preprint. arXiv:2308.09029 [cs.SE] doi:10.48550/arXiv.2308.09029

A Reproducibility Details

A.1 Color Processing and Metrics

Encoded sRGB inputs are decoded to linear sRGB before compositing [9]. Relative luminance is $Y(C) = 0.2126C_R + 0.7152C_G + 0.0722C_B$, and the contrast ratio is [8]

$$CR(C_o, C_b) = \frac{\max(Y_o, Y_b) + 0.05}{\min(Y_o, Y_b) + 0.05}.$$

The reference is the contrast ratio of the original source color against the same background. Changes with absolute magnitude at most 10^{-10} are counted as unchanged. Numerical calculations use 64-bit floating-point values. Difference uses $|C_s - C_b|$ and Exclusion uses $C_s + C_b - 2C_sC_b$, with componentwise operations in the same linear space.

Source and background grids are uniformly spaced in encoded sRGB, including both endpoints, and then decoded. The predefined set comprises eight light and eight dark canvases, including neutral and mildly tinted colors. Exact sRGB values are recorded in the experiment configuration.

For CIEDE2000, linear sRGB colors are converted to CIELAB using the sRGB D65 white point. Color-difference decreases use a tolerance of 10^{-9} . Collisions are evaluated after sRGB encoding and rounding to 8-bit values, with ties rounded to even. Random pairs use NumPy’s default generator with seed 20260913. The evaluation retains equal-color background cases and neutral source colors. It measures numerical color separation, not human recognition or full-page accessibility compliance.

A.2 Rendering and Timing

Icons come from lucide-static version 1.45.0. The corpus contains the first 300 SVG paths ordered by their SHA-256 path hashes, independently of rendering outcomes. Source colors cycle through the seven Vibrant colors. Browser validation uses 16, 24, and 48 px icons; the nine configurations are $\alpha \in \{0.6, 0.9, 1, 1.05, 1.1, 1.2, 1.4\}$, Difference, and Exclusion.

The shader obtains geometric coverage $q \in [0, 1]$ from the rasterized icon mask and computes

$$C_{\text{pixel}} = q \cdot \text{clip}_{[0,1]}(C_b + \alpha(C_s - C_b)) + (1 - q)C_b.$$

The result is encoded to sRGB for output. Coverage and the extended coefficient are separate, so antialiasing retains conventional edge coverage. The application supplies the already-composited

background; the prototype does not read arbitrary DOM backgrounds or change native CSS opacity semantics.

Timing uses a 1280×960 canvas with 300 icons at 48 px. Backgrounds alternate between a warm light canvas and a cool dark canvas. Each browser runs two warm-up sweeps, followed by 25 rounds over all 81 coefficients. A seeded Fisher-Yates permutation changes the order in each round; all browsers use the same schedule. Every frame ends with synchronous one-pixel readback. The elapsed `performance.now()` interval for 20 frames is divided by 20 to obtain a block mean. All 25 blocks per coefficient are retained. Reported times include draw submission and readback, rather than measuring GPU arithmetic alone.

A.3 Preservation of Background Distance

For each channel, write $s = C_{s,k}$ and $b = C_{b,k}$, with $s, b \in [0, 1]$ and $\alpha \geq 1$. If $s \geq b$, then $b + \alpha(s - b) \geq s$; clipping leaves the output in $[s, 1]$. If $s \leq b$, clipping leaves it in $[0, s]$. In both cases, $|C_{o,k} - b| \geq |s - b|$. Taking any ℓ_p norm gives the inequality in Section 3.4. This statement concerns real-valued RGB distances before output quantization.

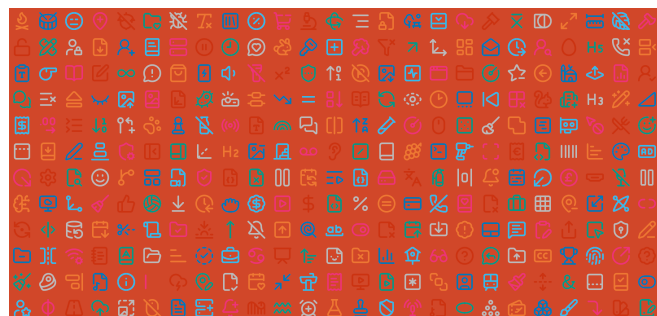
A.4 Visual Examples across Backgrounds

Figures 6 and 7 show all 300 icons on eight backgrounds: seven near-source colors obtained by mixing a Vibrant color with 10% white in encoded sRGB, and a dark grey canvas. Each pair compares $\alpha = 1$ and 2 with fixed source colors. These selected cases illustrate appearance rather than recognition performance.

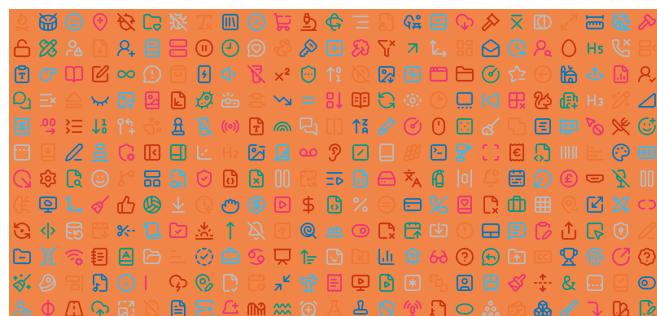
Original $\cdot \alpha = 1.0$

 $\alpha = 2.0$

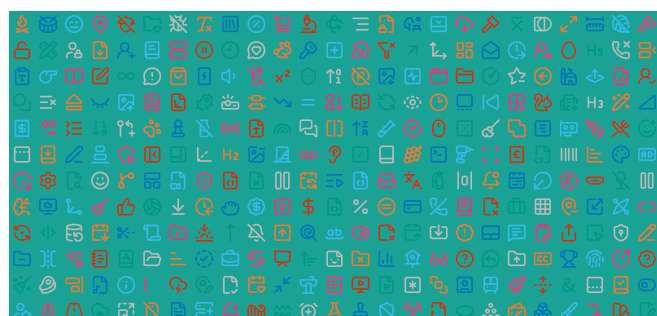
Red



Orange



Teal



Cyan

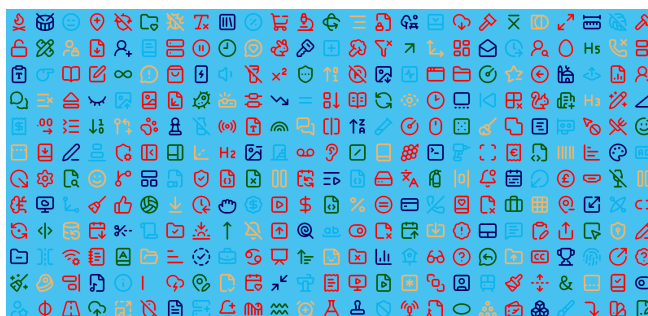
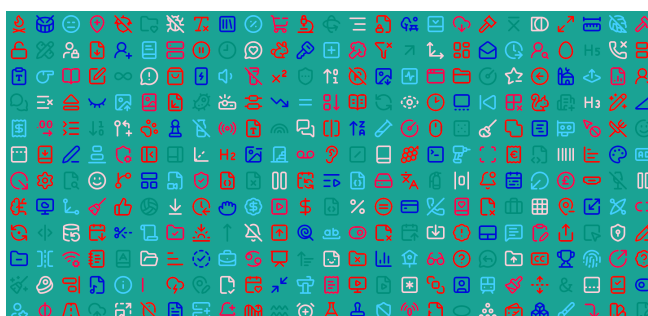
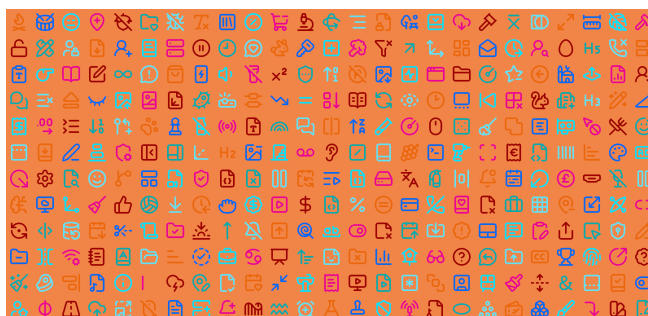
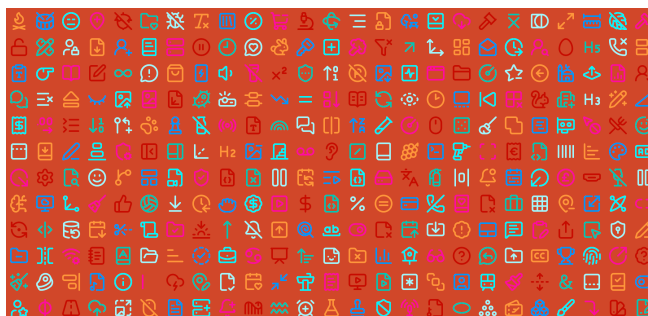
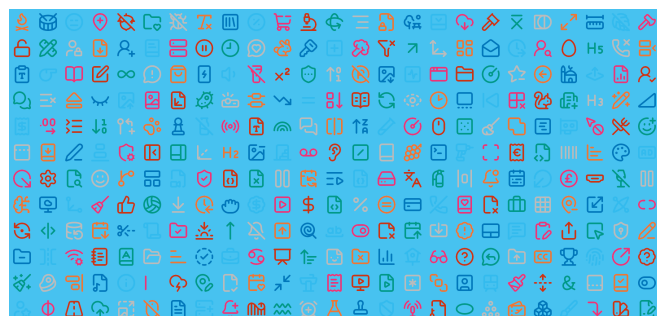


Figure 6: Full icon arrays on near-source backgrounds, ordered from warm to cool hues. Each row compares the same 300 icons at $\alpha = 1$ (left) and 2 (right). Source colors, icon order, and dimensions are fixed across all panels.

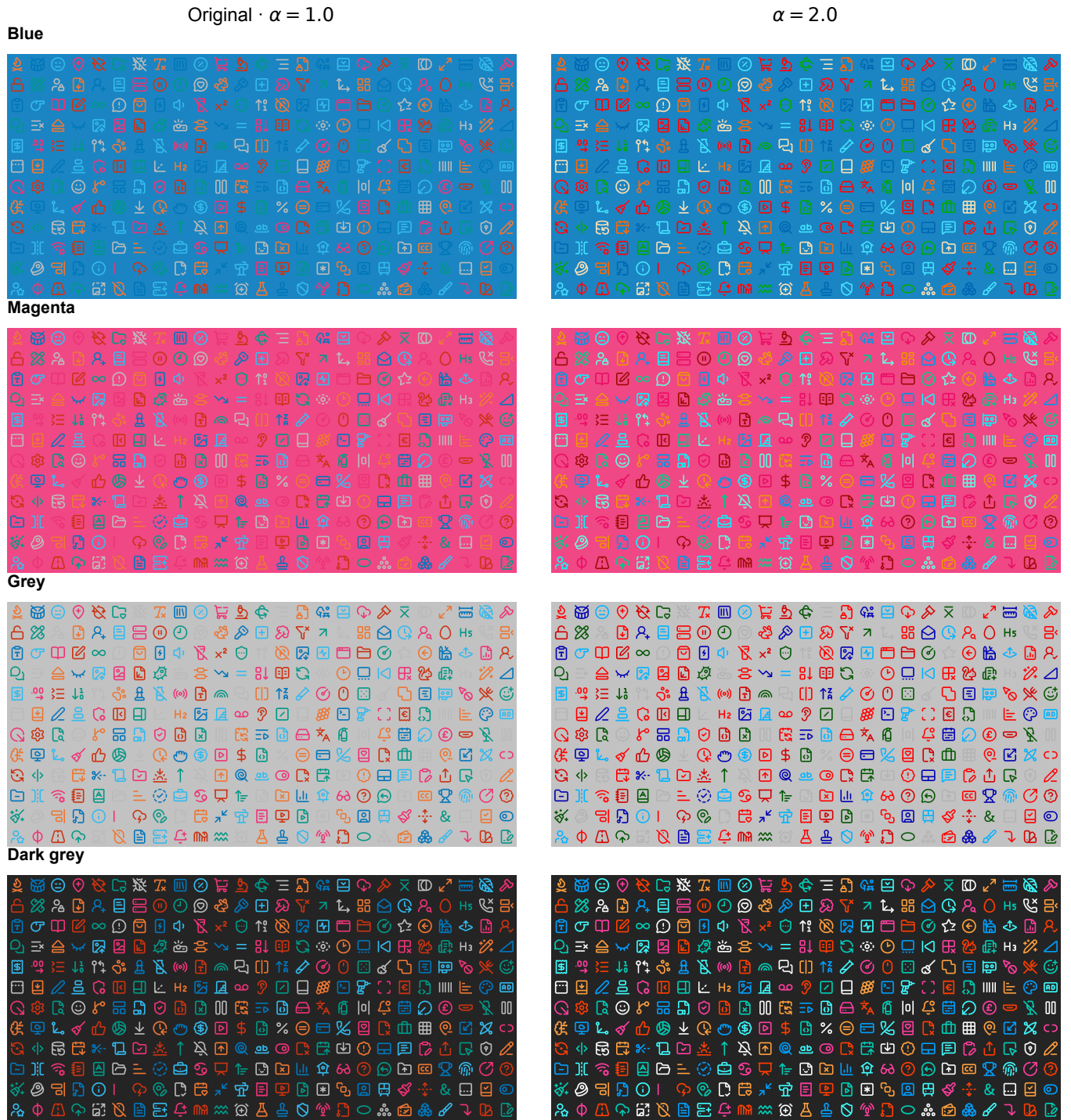


Figure 7: Continuation of the background comparisons: blue, magenta, grey, and dark grey. The same source colors and icon order are retained; each row compares $\alpha = 1$ (left) with 2 (right).